CMPE 223 / 242
Hands-On Activity 7

Assume that we have the following API for the Binary Tree, for the questions below.

```
public class BinarySearchTree {

  private Node root;

  private class Node {
     private int key;            % key
     private Node left;          % left child
     private Node right;         % right child
  }

}
```

1. Write a **recursive** Java solution to find the minimum key in the Binary Search Tree. The method should be implemented as a method of the `BinarySearchTree` class.

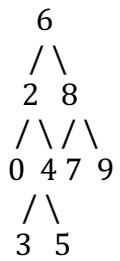
2. Write a **recursive** Java solution to print the keys in ascending order in the Binary Search Tree. The method should be implemented as a method of the `BinarySearchTree` class.

3. Write a **recursive** Java solution to find the height of the Binary Search Tree. The method should be implemented as a method of the `BinarySearchTree` class.

4. Write a **recursive** Java solution to delete the minimum key in the Binary Tree. The method should be implemented as a method of the `BinarySearchTree` class.

5. Given a Binary Search Tree (BST) and two nodes in the tree, find the lowest common ancestor (LCA) of the two nodes.

   The lowest common ancestor (LCA) of two nodes in a binary tree is the lowest (i.e., deepest) node that has both nodes as descendants. In a BST, the LCA of two nodes **p** and **q** would be the node where **p** and **q** diverge in their paths down the tree.



   For example, for the above BST:
   - the LCA of nodes 2 and 8 is node 6
   - the LCA of nodes 3 and 9 is node 6
   - the LCA of nodes 3 and 5 is node 4

   Input:
   - Root of the BST
   - Two nodes **p** and **q** for which you need to find the LCA.

   Output:
   - The lowest common ancestor node of **p** and **q**.

   Constraints:
   - All nodes' values are unique.
   - **p** and **q** are guaranteed to be present in the BST.

   Function Signature:

```
public Node lowestCommonAncestor(Node root, Node p, Node q)
```